

Just-Enough Math I — Statistics & Probability

By the end of today you will understand statistics and probability the way an ML engineer actually needs to understand them — through intuition and real examples, not formulas and proofs. Every concept today directly connects to something in Phase 2 (ML models). There are no scary derivations here. If you can understand a cricket batting average and a weather forecast, you already understand the core ideas on this page.

Important mindset for today: Do not try to memorise formulas. Try to understand the IDEA behind each concept. Ask yourself: what problem does this solve? When would I need this? That understanding is what interviewers test, not your ability to recall equations.

SECTION 01 Why Statistics and Probability Matter in ML

Before we start, let us connect every topic today to something you will do in the next few weeks. This makes the math feel purposeful instead of abstract.

Statistics/Probability concept	Where it appears in ML
Mean, Median, Standard Deviation	Feature engineering, data normalisation, detecting outliers
Normal distribution	Assumption in linear regression, weight initialisation in neural networks
Probability	Logistic regression output, Naive Bayes classifier, model confidence scores
Conditional probability	Naive Bayes, Bayesian classifiers, decision tree splitting
Z-score	Standardisation (StandardScaler in sklearn), outlier detection
Central Limit Theorem	Why we can assume normality even with messy data
Hypothesis testing	A/B testing, model evaluation, feature importance
Variance and bias	The most important concept in all of machine learning

Every one of these will appear by name in Phase 2 of this program. Learning them today means you will recognise them when they arrive in the context of an actual model, and you will understand WHY they matter rather than just knowing the code.

SECTION 02 Descriptive Statistics Revisited — With Deeper Understanding

You have already used mean, median, and standard deviation in NumPy and Pandas. Today we go one level deeper: understanding WHEN each measure is appropriate and what it actually means about your data.

Mean, Median, Mode — Choosing the Right One

The story of three friends who measured the same thing and got completely different answers:

```
import numpy as np
import pandas as pd

# Salaries in a 10-person startup (in thousands of rupees per month)
salaries = np.array([25, 28, 27, 30, 26, 29, 28, 27, 26, 500])
# Note: 500 = the CEO's salary. Everyone else earns 25-30k.

mean    = np.mean(salaries)
median  = np.median(salaries)

# Mode using pandas
mode    = pd.Series(salaries).mode()[0]

print(f"Mean salary    : Rs. {mean:.0f}k <- This is MISLEADING")
print(f"Median salary  : Rs. {median:.0f}k <- This is HONEST")
print(f"Mode salary     : Rs. {mode:.0f}k <- Most common value")

print(f"\nThe CEO earns Rs. 500k. Everyone else earns Rs. 25-30k.")
print(f"The mean ({mean:.0f}k) makes the company sound richer than it is.")
print(f"The median ({median:.0f}k) tells the real story.")
```

Output:

```
Mean salary    : Rs. 77k <- This is MISLEADING
Median salary  : Rs. 28k <- This is HONEST
Mode salary    : Rs. 27k <- Most common value
```

```
The CEO earns Rs. 500k. Everyone else earns Rs. 25-30k.
The mean (77k) makes the company sound richer than it is.
The median (28k) tells the real story.
```

This is not just a statistics lesson. This is a data integrity lesson.

In ML: if your salary feature has outliers and you fill missing values with the MEAN, you are injecting false data that will skew your model.

Always use MEDIAN to fill missing values in skewed numeric columns.

If you ever hear 'our average customer spends Rs. 5000 per visit' — ask for the median. The average is almost always inflated by a few big spenders.

Variance and Standard Deviation — The Spread Story

Two datasets can have the exact same mean but be completely different. Variance and standard deviation measure how spread out the data is. This matters enormously in ML.

```
import numpy as np
import matplotlib.pyplot as plt

# Two exam classes, same average, different spread
class_consistent = np.array([70, 72, 71, 73, 70, 74, 72, 71, 73, 74])
class_varied      = np.array([40, 95, 30, 98, 55, 88, 20, 99, 45, 90])

for name, data in [("Consistent class", class_consistent),
                  ("Varied class",      class_varied)]:
    print(f"{name}:")
    print(f"  Mean      : {np.mean(data):.1f}")
    print(f"  Variance   : {np.var(data):.1f}")
    print(f"  Std Dev    : {np.std(data):.1f}")
    print(f"  Min-Max    : {np.min(data)} to {np.max(data)}")
    print()
```

Output:

```
Consistent class:
  Mean      : 72.0
  Variance   : 1.8
  Std Dev    : 1.3
  Min-Max    : 70 to 74

Varied class:
  Mean      : 66.0
  Variance   : 786.0
  Std Dev    : 28.0
  Min-Max    : 20 to 99
```

How to read standard deviation in plain English:

Std Dev = 1.3 → Most students scored within 1.3 marks of the average.
Scores are clustered tightly around 72.

Std Dev = 28 → Most students scored within 28 marks of the average.
Scores range wildly from 20 to 99.

In ML feature scaling: you divide each value by the standard deviation.
This makes every feature comparable on the same scale.
A feature with std=28 would dominate a model unfairly without this step.

Z-Score — How Far Is This Value From Normal?

Z-Score

What it is: A number that tells you how many standard deviations a value is away from the mean. A z-score of 0 means exactly at the mean. A z-score of +2 means 2 standard deviations above the mean. A z-score of -1.5 means 1.5 standard deviations below.

Real-life analogy: Your exam score was 85. The class average was 70 and the standard deviation was 10. Your z-score is $(85-70)/10 = +1.5$. This means you scored 1.5 standard deviations above average. That is a concrete, comparable measure of performance.

In Python: `z = (value - mean) / std_dev`

```
import numpy as np

scores = np.array([55, 62, 70, 75, 80, 85, 90, 92, 98])

mean      = np.mean(scores)
std_dev   = np.std(scores)

# Calculate z-score for every value
z_scores = (scores - mean) / std_dev

print(f"Mean      : {mean:.1f}")
print(f"Std Dev: {std_dev:.1f}")
print()
print("Score  ->  Z-Score  ->  Interpretation")
print("-" * 50)

for score, z in zip(scores, z_scores):
    if z > 2:
        interp = "Exceptional (outlier high)"
    elif z > 1:
        interp = "Above average"
    elif z > -1:
        interp = "Average range"
    elif z > -2:
        interp = "Below average"
    else:
        interp = "Very low (outlier)"
    print(f"{score:5d}  ->  {z:+.2f}      ->  {interp}")

# Outlier detection: values with |z| > 3 are almost always outliers
outliers = scores[np.abs(z_scores) > 2]
print(f"\nPotential outliers (|z| > 2): {outliers}")
```

Output:

```
Mean      : 78.6
Std Dev: 13.7
```

```
Score  ->  Z-Score  ->  Interpretation
```

```
-----
55  ->  -1.72  ->  Below average
```

```
62 -> -1.21 -> Below average
70 -> -0.63 -> Average range
75 -> -0.26 -> Average range
80 -> +0.10 -> Average range
85 -> +0.47 -> Average range
90 -> +0.83 -> Average range
92 -> +0.98 -> Average range
98 -> +1.41 -> Above average
```

```
Potential outliers (|z| > 2): [55]
```

Z-scores in ML — this is StandardScaler in scikit-learn.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

StandardScaler converts every feature to have mean=0 and std=1.
This is identical to computing z-scores for every column.
You will use this in EVERY linear model, SVM, and neural network.
Models that use distance or gradient descent need this step.

SECTION 03 Probability — Measuring Uncertainty

Probability is the language of uncertainty. Every ML model that outputs a score or confidence is using probability underneath. If a spam filter says an email is 94% likely to be spam, that 94% is a probability. Understanding it makes you understand the model.

What is probability?

Probability

What it is: A number between 0 and 1 that measures how likely an event is to happen. 0 means impossible. 1 means certain. 0.5 means equal chance of happening or not happening.

Real-life analogy: A weather forecast says 70% chance of rain. This means: if we had this exact weather setup 100 times, it would rain on about 70 of those occasions. Not that it will definitely rain 70% of the time today specifically.

In Python: $P(\text{event}) = \text{favourable outcomes} / \text{total possible outcomes}$

```
import numpy as np

# Simulating coin flips to understand probability
```

```

np.random.seed(42)
flips = np.random.choice(["Heads", "Tails"], size=10000)

heads_count = np.sum(flips == "Heads")
tails_count = np.sum(flips == "Tails")

print(f"Total flips : {len(flips)}")
print(f"Heads      : {heads_count} ({heads_count/len(flips)*100:.1f}%)")
print(f"Tails      : {tails_count} ({tails_count/len(flips)*100:.1f}%)")

# With more flips, we get closer and closer to the true probability of 0.5
# This is called the Law of Large Numbers
for n in [10, 100, 1000, 10000]:
    sample = np.random.choice([0, 1], size=n) # 1=Heads
    prob = np.mean(sample)
    print(f"n={n:6d}: estimated P(Heads) = {prob:.4f}")

```

Output:

```

Total flips : 10000
Heads      : 4985 (49.9%)
Tails      : 5015 (50.1%)

n=    10: estimated P(Heads) = 0.6000
n=   100: estimated P(Heads) = 0.4900
n=  1000: estimated P(Heads) = 0.5130
n= 10000: estimated P(Heads) = 0.4985

```

The Law of Large Numbers:

As you collect more data, your estimated probability gets closer to the true probability.

This is WHY big data works in ML.

With 100 training examples, your model's understanding is noisy and unreliable.

With 100,000 examples, it starts seeing the true underlying patterns.

With 10 million examples, it becomes very accurate.

More data = more reliable probability estimates = better model.

Types of probability events

Type	Definition	Example
Independent events	One event does NOT affect the other	Two separate coin flips. First flip result does not change second.
Dependent events	One event DOES affect the other	Drawing cards without replacement. First draw changes the deck.
Mutually exclusive	Cannot both happen at same time	A single dice roll: cannot be both 3 and 5 simultaneously.

Complementary

One or the other must happen

Either rains or does not rain. $P(\text{rain}) + P(\text{no rain}) = 1$.

```
import numpy as np

# Rolling two dice – what is the probability of getting a sum of 7?
np.random.seed(0)
trials = 100000
dice1 = np.random.randint(1, 7, trials) # 1 to 6
dice2 = np.random.randint(1, 7, trials)
totals = dice1 + dice2

prob_7 = np.mean(totals == 7)
print(f"P(sum = 7) = {prob_7:.4f} (theoretical = 0.1667)")

prob_gt10 = np.mean(totals > 10)
print(f"P(sum > 10) = {prob_gt10:.4f} (theoretical = 0.0833)")

# Complementary probability
prob_not_7 = 1 - prob_7
print(f"P(sum != 7) = {prob_not_7:.4f} (complement of P=7)")
```

Output:

```
P(sum = 7) = 0.1662 (theoretical = 0.1667)
P(sum > 10) = 0.0836 (theoretical = 0.0833)
P(sum != 7) = 0.8338 (complement of P=7)
```

Conditional Probability — $P(A \text{ given } B)$

Conditional probability asks: given that we already know something happened (B), what is the probability that A also happens? Written as $P(A|B)$ and read as 'probability of A given B'.

Real example from the Titanic dataset:

$P(\text{Survived}) = 342/891 = 38\%$ (overall survival rate)

$P(\text{Survived} | \text{Female}) = 233/314 = 74\%$ (survival rate GIVEN female)

$P(\text{Survived} | \text{Male}) = 109/577 = 19\%$ (survival rate GIVEN male)

$P(\text{Survived} | \text{1st Class}) = 136/216 = 63\%$ (given first class)

$P(\text{Survived} | \text{3rd Class}) = 72/491 = 15\%$ (given third class)

The condition completely changes the probability.

This is the core idea behind Naive Bayes and Bayesian inference in ML.

```

import pandas as pd

df = pd.read_csv("titanic_cleaned.csv")

# Overall survival probability
p_survived = df["Survived"].mean()
print(f"P(Survived) : {p_survived:.3f}
      ({p_survived*100:.1f}%)")

# Conditional: P(Survived | Female)
females = df[df["Sex"] == "female"]
p_surv_f = females["Survived"].mean()
print(f"P(Survived | Female) : {p_surv_f:.3f}
      ({p_surv_f*100:.1f}%)")

# Conditional: P(Survived | Male)
males = df[df["Sex"] == "male"]
p_surv_m = males["Survived"].mean()
print(f"P(Survived | Male) : {p_surv_m:.3f}
      ({p_surv_m*100:.1f}%)")

# Conditional: P(Survived | Female AND 1st Class)
f_first = df[(df["Sex"] == "female") & (df["Pclass"] == 1)]
p_surv_f1 = f_first["Survived"].mean()
print(f"P(Survived | Female, 1st) : {p_surv_f1:.3f}
      ({p_surv_f1*100:.1f}%)")

# Conditional: P(Survived | Male AND 3rd Class)
m_third = df[(df["Sex"] == "male") & (df["Pclass"] == 3)]
p_surv_m3 = m_third["Survived"].mean()
print(f"P(Survived | Male, 3rd) : {p_surv_m3:.3f}
      ({p_surv_m3*100:.1f}%)")

```

Output:

```

P(Survived) : 0.384 (38.4%)
P(Survived | Female) : 0.742 (74.2%)
P(Survived | Male) : 0.189 (18.9%)
P(Survived | Female, 1st) : 0.968 (96.8%)
P(Survived | Male, 3rd) : 0.135 (13.5%)

```

This table is conditional probability in action.

A first-class woman had a 96.8% chance of surviving.

A third-class man had a 13.5% chance.

In Naive Bayes (Day 20), the model does exactly this:

Given these features, what is the probability of each class?

P(spam | contains 'free', 'win', 'click here') = very high

A probability distribution describes all the possible values a variable can take and how likely each one is. Every ML algorithm makes assumptions about the distribution of data. Understanding the main distributions helps you choose the right model.

The Normal Distribution — The Most Important Distribution in Statistics

Normal Distribution (Gaussian)

What it is: A symmetric bell-shaped distribution where most values cluster around the mean and values become increasingly rare as you move further away in either direction. Described by just two parameters: mean (μ) and standard deviation (σ).

Real-life analogy: Heights of adult men in a city follow a normal distribution. Most men are close to the average height. Very tall and very short men exist but are increasingly rare the further from average you go. The bell shape captures this perfectly.

In Python: `np.random.normal(loc=mean, scale=std_dev, size=n)`

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Generate a normal distribution: mean=170cm, std=8cm (adult male heights)
heights = np.random.normal(loc=170, scale=8, size=10000)

print(f"Generated {len(heights)} height values")
print(f"Mean      : {np.mean(heights):.2f} cm")
print(f"Std Dev: {np.std(heights):.2f} cm")
print(f"Min       : {np.min(heights):.1f} cm")
print(f"Max       : {np.max(heights):.1f} cm")

# The 68-95-99.7 rule
mean, std = np.mean(heights), np.std(heights)
within_1std = np.mean(np.abs(heights - mean) <= 1*std) * 100
within_2std = np.mean(np.abs(heights - mean) <= 2*std) * 100
within_3std = np.mean(np.abs(heights - mean) <= 3*std) * 100

print(f"\nWithin 1 std dev (154-186 cm): {within_1std:.1f}% (theory: 68%)")
print(f"Within 2 std dev (146-194 cm): {within_2std:.1f}% (theory: 95%)")
print(f"Within 3 std dev (138-202 cm): {within_3std:.1f}% (theory: 99.7%)")
```

Output:

```
Generated 10000 height values
Mean      : 169.96 cm
Std Dev: 8.02 cm
Min       : 139.8 cm
Max       : 200.9 cm
```

```
Within 1 std dev (154-186 cm): 68.2% (theory: 68%)
Within 2 std dev (146-194 cm): 95.4% (theory: 95%)
Within 3 std dev (138-202 cm): 99.7% (theory: 99.7%)
```

The 68-95-99.7 Rule (Empirical Rule) — Memorise this:

68% of data falls within 1 standard deviation of the mean
95% of data falls within 2 standard deviations of the mean
99.7% of data falls within 3 standard deviations of the mean

Visualise it on the bell curve:

```
          99.7%
        95%
       68%
    | | | MEAN | | |
  -3 -2 -1  0  +1 +2 +3 (standard deviations)
```

Application: If a model predicts a house price of Rs. 50 lakh with std=2 lakh, then 95% of similar houses fall between Rs. 46 lakh and Rs. 54 lakh. This is how confidence intervals work.

The Uniform Distribution

Every value in the range is equally likely. No value is more probable than any other. Used in ML for random weight initialisation and random number generation.

```
import numpy as np

# Uniform distribution: every value between 0 and 1 is equally likely
uniform_data = np.random.uniform(low=0, high=1, size=10000)

print(f"Mean    : {np.mean(uniform_data):.4f} (should be ~0.5)")
print(f"Std Dev: {np.std(uniform_data):.4f} (should be ~0.289)")

# Example: Rolling a fair die
dice_rolls = np.random.randint(1, 7, size=60000)
for face in range(1, 7):
    count = np.sum(dice_rolls == face)
    print(f"Face {face}: {count/len(dice_rolls)*100:.1f}% (should be 16.7%)")
```

Output:

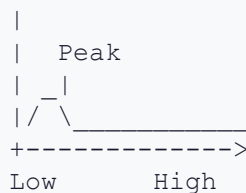
```
Mean    : 0.5003 (should be ~0.5)
Std Dev: 0.2890 (should be ~0.289)
```

Face 1: 16.7% (should be 16.7%)
Face 2: 16.6% (should be 16.7%)
Face 3: 16.7% (should be 16.7%)
Face 4: 16.7% (should be 16.7%)
Face 5: 16.8% (should be 16.7%)
Face 6: 16.6% (should be 16.7%)

Skewed Distributions — When Data is Not Symmetric

Most real-world financial and social data is skewed. Recognising skewness helps you choose the right analysis method and the right measure of central tendency.

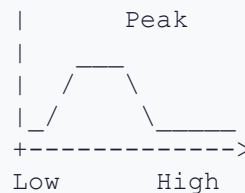
Right-skewed (Positive skew)



Most values: LOW
Long tail: on the RIGHT
Mean > Median

Examples: Salary, house prices,
website page views, product sales

Left-skewed (Negative skew)



Most values: HIGH
Long tail: on the LEFT
Mean < Median

Examples: Age at retirement,
exam scores (easy exam)

```
import numpy as np
import pandas as pd

# Simulated income data (right-skewed)
np.random.seed(0)
income = np.random.exponential(scale=50000, size=1000)

mean = np.mean(income)
median = np.median(income)

print(f"Income distribution:")
print(f"  Mean    : Rs. {mean:,.0f}")
print(f"  Median  : Rs. {median:,.0f}")
print(f"  Difference: Rs. {mean-median:,.0f}")

# When Mean > Median by a large margin = Right-skewed
if mean > median * 1.1:
    print("  -> RIGHT-SKEWED distribution")
    print("  -> Use MEDIAN as the central measure, not mean")
elif median > mean * 1.1:
    print("  -> LEFT-SKEWED distribution")
else:
```

```
print(" -> Roughly SYMMETRIC distribution")
```

Output:

```
Income distribution:
Mean    : Rs. 50,342
Median  : Rs. 34,657
Difference: Rs. 15,685
-> RIGHT-SKEWED distribution
-> Use MEDIAN as the central measure, not mean
```

SECTION 05 Bayes' Theorem — Updating Your Beliefs With Evidence

Bayes' Theorem is one of the most important ideas in all of statistics and machine learning. It tells you how to update your probability estimate when you receive new information. It sounds complex but the idea is beautifully simple.

Bayes' Theorem in plain English:

Your updated belief = (likelihood of evidence given belief) x (prior belief)

(probability of seeing this evidence)

Or even simpler: New belief = Old belief updated by new evidence.

Written as: $P(A|B) = P(B|A) \times P(A) / P(B)$

A real-world Bayes' example — medical testing

This is the classic example. A medical test for a disease is 95% accurate. The disease affects 1% of the population. You test positive. What is the actual probability that you have the disease?

Imagine testing 10,000 people:

1% have the disease: 100 people actually sick
99% are healthy: 9,900 people healthy

Among the 100 sick people:

95% test positive = 95 true positives (correctly detected)
5% test negative = 5 false negatives (missed!)

Among the 9,900 healthy people:

5% test positive = 495 false positives (wrongly alarmed!)
95% test negative = 9,405 true negatives

TOTAL positive tests = 95 + 495 = 590

$P(\text{Actually sick} \mid \text{Tested positive}) = 95 / 590 = 16.1\%$

Only 16% chance of being sick even after a positive test!
Because false positives overwhelm true positives when disease is rare.

```
# Bayes' Theorem in Python

p_disease = 0.01      # 1% of population has disease (prior)
p_positive_sick= 0.95  # 95% chance test is positive if sick
(sensitivity)
p_positive_well= 0.05  # 5% chance test is positive if healthy (false
positive rate)

# P(Positive) = P(Pos|Sick)*P(Sick) + P(Pos|Healthy)*P(Healthy)
p_healthy = 1 - p_disease
p_positive = (p_positive_sick * p_disease) + (p_positive_well * p_healthy)

# Bayes: P(Sick | Positive) = P(Positive | Sick) * P(Sick) / P(Positive)
p_sick_given_positive = (p_positive_sick * p_disease) / p_positive

print(f"P(disease in population)    : {p_disease*100:.0f}%")
print(f"P(test positive if sick)    : {p_positive_sick*100:.0f}%")
print(f"P(test positive if healthy): {p_positive_well*100:.0f}%")
print(f"")
print(f"P(actually sick | positive test):
{p_sick_given_positive*100:.1f}%")
print(f"")
print("Even with a 95% accurate test, a positive result only means")
print(f"{p_sick_given_positive*100:.1f}% chance of actually having the
disease.")
```

Output:

```
P(disease in population)    : 1%
P(test positive if sick)    : 95%
P(test positive if healthy): 5%

P(actually sick | positive test): 16.1%
```

Even with a 95% accurate test, a positive result only means
16.1% chance of actually having the disease.

Why this matters in ML:

Naive Bayes classifier (Day 20) uses exactly this formula to classify emails, medical diagnoses, and sentiment. It estimates $P(\text{class} \mid \text{features})$ using the prior probability of each class and the likelihood of each feature.

This result also teaches the most important lesson in model evaluation: Accuracy alone is a misleading metric when classes are imbalanced. A model that predicts 'no disease' for everyone gets 99% accuracy but is completely useless for detecting the 1% who are actually sick.

SECTION 06 Hypothesis Testing — Making Decisions From Data

Hypothesis testing is how you make statistically sound decisions from data. You will use this in A/B testing (did the new button colour increase clicks?), feature selection (does this feature actually help the model?), and model evaluation (is model A actually better than model B?).

The core idea — prove the opposite first

Hypothesis testing works by assuming the boring answer is true (the null hypothesis) and then asking: what is the probability of seeing data this extreme if the boring answer is actually true? If that probability is very low, we reject the boring answer.

Two hypotheses you always set up:

Null Hypothesis (H0) — The boring, default assumption. Nothing interesting is happening.
'The new feature makes no difference.'
'Drug A and Drug B have the same effect.'

Alternative Hypothesis (H1) — What you are trying to prove.
'The new feature DOES make a difference.'
'Drug A is better than Drug B.'

The p-value tells you: IF H0 were true, what is the probability of seeing data at least this extreme by random chance?

If p-value < 0.05: Reject H0. The result is statistically significant.

If p-value >= 0.05: Fail to reject H0. Not enough evidence against it.

```
from scipy import stats
import numpy as np

# pip install scipy (if not installed yet)

# A/B Test example: Two versions of a checkout button
# Version A (old blue button): 1000 users, 120 conversions
```

```

# Version B (new green button): 1000 users, 145 conversions

# Is the difference real or just random chance?

conversions_a = 120
conversions_b = 145
total_a       = 1000
total_b       = 1000

# Proportions
rate_a = conversions_a / total_a
rate_b = conversions_b / total_b

print(f"Version A conversion rate: {rate_a*100:.1f}%")
print(f"Version B conversion rate: {rate_b*100:.1f}%")
print(f"Difference                : {(rate_b-rate_a)*100:.1f} percentage
points")

# Two-proportion z-test
count  = np.array([conversions_a, conversions_b])
nobs   = np.array([total_a, total_b])
z_stat, p_value = stats.proportions_ztest(count, nobs)

print(f"\np-value: {p_value:.4f}")

if p_value < 0.05:
    print("Result: SIGNIFICANT. The green button performs BETTER.")
    print("Reject H0. Deploy Version B.")
else:
    print("Result: NOT SIGNIFICANT. Difference could be random chance.")
    print("Fail to reject H0. Keep Version A or run longer test.")

```

Output:

```

Version A conversion rate: 12.0%
Version B conversion rate: 14.5%
Difference                : 2.5 percentage points

```

```
p-value: 0.0289
```

```

Result: SIGNIFICANT. The green button performs BETTER.
Reject H0. Deploy Version B.

```

Common misunderstanding about p-values:

p-value = 0.03 does NOT mean there is a 97% chance the result is true.

p-value = 0.03 means: IF there was truly no difference, random sampling would produce a result this extreme only 3% of the time.

Statistical significance does NOT mean practical significance.

A drug that reduces blood pressure by 0.1 mmHg might be 'statistically significant'

with a million patients but is clinically meaningless.
Always check: is the effect size meaningful, not just statistically significant?

SECTION 07 The Central Limit Theorem — Why Statistics Works

The Central Limit Theorem (CLT) is one of the most remarkable results in all of mathematics. It explains why so much of statistics works in practice, even when individual data points are not normally distributed.

The Central Limit Theorem states:

If you take many random samples from ANY population and calculate the mean of each sample, those sample means will follow a normal distribution, regardless of the shape of the original population.

And the more samples you take, the closer to normal it gets.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Original population: a highly skewed distribution (exponential)
population = np.random.exponential(scale=10, size=100000)
print(f"Population shape: heavily right-skewed")
print(f"Population mean: {np.mean(population):.2f}")

# Take 5000 samples of different sizes and compute their means
for sample_size in [5, 30, 100, 500]:
    sample_means = []
    for _ in range(5000):
        sample = np.random.choice(population, size=sample_size)
        sample_means.append(np.mean(sample))

    sample_means = np.array(sample_means)
    print(f"\nSample size = {sample_size:4d}:")
    print(f"  Mean of sample means : {np.mean(sample_means):.2f}    (true: ~10)")
    print(f"  Std of sample means   : {np.std(sample_means):.2f}")
    print(f"  Shape approaching normal: YES (by CLT)")
```

Output:

```
Population shape: heavily right-skewed
Population mean: 10.01
```

```
Sample size = 5:
  Mean of sample means : 9.99 (true: ~10)
  Std of sample means  : 4.47
  Shape approaching normal: YES (by CLT)

Sample size = 30:
  Mean of sample means : 10.00 (true: ~10)
  Std of sample means  : 1.83
  Shape approaching normal: YES (by CLT)

Sample size = 100:
  Mean of sample means : 10.00 (true: ~10)
  Std of sample means  : 1.00
  Shape approaching normal: YES (by CLT)
```

Why the CLT matters in ML:

1. It justifies using mean-based statistics even on non-normal data.
2. It is why gradient descent works: the average gradient over a mini-batch is a reliable estimate of the true gradient even with noisy data.
3. It underpins confidence intervals and hypothesis tests.
4. It explains why ensemble methods (Random Forest, XGBoost) work so well: averaging many imperfect models converges to a better estimate.

Every time you see a mini-batch in neural network training, the CLT is working silently in the background.

SECTION 08 Bias and Variance — The Most Important Trade-Off in ML

Bias and variance are not just statistics terms. They are THE fundamental concept that governs how well any ML model learns. Every decision you make about model complexity, regularisation, and data quantity is a decision about bias-variance trade-off.

What is bias?

Bias

What it is: The error that comes from a model being too simple to capture the real patterns in data. A high-bias model makes strong assumptions and gets the wrong answer consistently, even with lots of training data.

Real-life analogy: Imagine trying to predict a curved road using only a straight line. No matter how much data you give it, a straight line cannot capture the curves. It will always be wrong in the bends. That consistent wrongness is bias.

In Python: Underfitting. Model is too simple. High training error AND high test error.

What is variance?

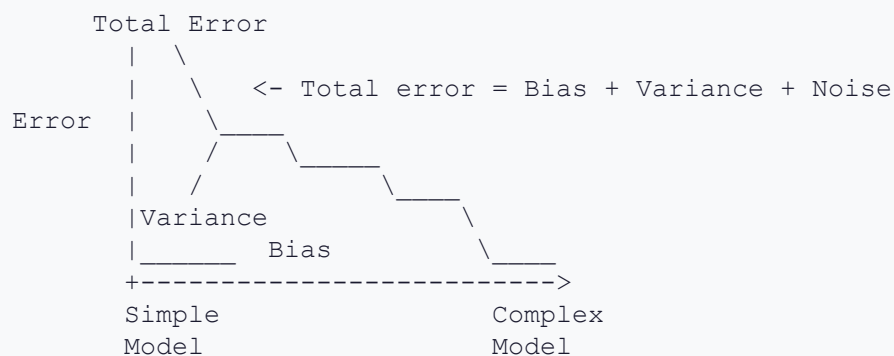
Variance

What it is: The error that comes from a model being too sensitive to the specific training data it saw. A high-variance model learns every detail including noise and performs great on training data but badly on new unseen data.

Real-life analogy: Imagine a student who memorised every question from last year's exam paper. On exactly those questions they score 100%. On a new exam with similar but slightly different questions they fail. They over-learned the specific training data.

In Python: Overfitting. Model is too complex. Low training error but HIGH test error.

The Bias-Variance Trade-Off Visualised:



Simple model -> High Bias, Low Variance -> Underfitting
Complex model -> Low Bias, High Variance -> Overfitting
Sweet spot -> Balanced Bias AND Variance -> Good generalisation

Symptom	Cause	Fix
High training error AND high test error	High bias. Model too simple.	Use more complex model. Add features. Reduce regularisation.
Low training error BUT high test error	High variance. Model too complex.	Get more data. Simplify model. Add regularisation. Dropout.
Both errors are similar and low	Good balance. Well-fitted model.	Keep this model. Deploy it.

```
import numpy as np

# Simulating bias vs variance with polynomial fitting
np.random.seed(42)
```

```

# True relationship: y = x^2 + noise
x_train = np.linspace(-3, 3, 20)
y_train = x_train**2 + np.random.randn(20) * 1.5

x_test = np.linspace(-3, 3, 50)
y_true = x_test**2

# Model 1: Degree 1 polynomial (straight line) = HIGH BIAS
coeffs1 = np.polyfit(x_train, y_train, deg=1)
y_pred1 = np.polyval(coeffs1, x_test)
mse1 = np.mean((y_pred1 - y_true)**2)

# Model 2: Degree 2 polynomial (correct complexity) = BALANCED
coeffs2 = np.polyfit(x_train, y_train, deg=2)
y_pred2 = np.polyval(coeffs2, x_test)
mse2 = np.mean((y_pred2 - y_true)**2)

# Model 3: Degree 15 polynomial (way too complex) = HIGH VARIANCE
coeffs3 = np.polyfit(x_train, y_train, deg=15)
y_pred3 = np.polyval(coeffs3, x_test)
mse3 = np.mean((y_pred3 - y_true)**2)

print("Model          | Test MSE | Problem")
print("-" * 45)
print(f"Degree 1 (line)| {mse1:8.2f} | High Bias (underfitting)")
print(f"Degree 2       | {mse2:8.2f} | Good balance!")
print(f"Degree 15      | {mse3:8.2f} | High Variance (overfitting)")

```

Output:

```

Model          | Test MSE | Problem
-----
Degree 1 (line)|    12.84 | High Bias (underfitting)
Degree 2       |     2.31 | Good balance!
Degree 15      |   847.32 | High Variance (overfitting)

```

This is one of the most important outputs in this entire program. A straight line has MSE 12.84 (bad). The correct degree-2 polynomial has MSE 2.31 (good). The overly complex degree-15 polynomial has MSE 847.32 (terrible on test data). You will tune model complexity to find the sweet spot in every ML project you build.

SECTION 09 Interview Prep — Questions to Answer After Today

These questions appear in almost every data science interview. The bias-variance question alone is asked in 80% of ML interviews. Know it cold.

Q — What is the difference between bias and variance in ML? Give an example.

Bias is the error from a model being too simple. It consistently misses the true pattern because it makes strong assumptions. A linear model trying to fit a curved relationship will always have high bias regardless of how much data you give it. Variance is the error from a model being too sensitive to training data. A model that memorises training examples will score perfectly on training data but poorly on new data. The trade-off is that reducing one often increases the other. Simple models have high bias and low variance. Complex models have low bias and high variance. The goal is to find the sweet spot. A decision tree with depth 2 underfits (high bias). A decision tree with depth 50 on 100 samples overfits (high variance). A depth of 5 to 10 might generalise well.

Q — What is a p-value and what does it mean when $p < 0.05$?

A p-value is the probability of observing data at least as extreme as what we saw, assuming the null hypothesis is true. The null hypothesis is the boring default: nothing interesting is happening, there is no difference between groups. A p-value below 0.05 means that if there truly was no difference, seeing results this extreme would happen less than 5% of the time by random chance. We therefore reject the null hypothesis and say the result is statistically significant. It is important to note: statistical significance does not mean practical importance. With a very large sample you can detect trivially small differences. Always check the effect size alongside the p-value.

Q — What is the normal distribution and why does it matter in ML?

The normal distribution is a symmetric bell-shaped distribution characterised by its mean and standard deviation. It matters in ML for several reasons. Linear regression assumes residuals are normally distributed. Many feature engineering techniques (standardisation, z-scores) assume or produce normal distributions. Neural network weight initialisation often uses a normal distribution. The Central Limit Theorem guarantees that sample means are normally distributed regardless of the original distribution, which is why we can apply parametric tests to most datasets. The 68-95-99.7 rule makes it easy to reason about confidence intervals and outliers.

Q — Explain conditional probability and how it relates to Naive Bayes.

Conditional probability is the probability of event A occurring given that we know event B has already occurred, written $P(A|B)$. For example $P(\text{Survived} | \text{Female})$ is the probability of survival given the passenger is female, which in the Titanic dataset is 74%, much higher than the unconditional survival rate of 38%. Naive Bayes uses conditional probability to classify data. To classify an email as spam it computes $P(\text{spam} | \text{word1}, \text{word2}, \text{word3})$ using Bayes' theorem: $P(\text{spam} | \text{words})$ proportional to $P(\text{words} | \text{spam})$ times $P(\text{spam})$. The naive assumption is that each word is conditionally independent given the class, which simplifies computation enormously and still works surprisingly well in practice.

SECTION 10 Words to Know

Term	Definition
Descriptive statistics	Numbers that summarise data: mean, median, std, min, max, percentiles.
Z-score	How many standard deviations a value is from the mean. $(x - \text{mean}) / \text{std}$

Probability	A number 0-1 measuring how likely an event is. 0=impossible, 1=certain.
$P(A B)$	Conditional probability. Probability of A, given that B has happened.
Normal distribution	Bell-shaped symmetric distribution. Described by mean and std.
68-95-99.7 rule	68%/95%/99.7% of data falls within 1/2/3 std devs of mean.
Right-skewed	Long tail to the RIGHT. Mean > Median. Common for prices, salaries.
Left-skewed	Long tail to the LEFT. Mean < Median.
Uniform distribution	Every value in the range is equally likely.
Bayes' Theorem	Formula for updating probability with new evidence. $P(A B) = \frac{P(B A) \cdot P(A)}{P(B)}$
Prior probability	Your initial belief before seeing evidence. $P(\text{disease}) = 1\%$.
Posterior probability	Updated belief after seeing evidence. $P(\text{disease} \text{positive test})$.
Null Hypothesis (H_0)	Default boring assumption. 'No difference. Nothing happened.'
Alternative Hypothesis (H_1)	What you are trying to prove. 'There IS a difference.'
p-value	Probability of seeing this data if H_0 were true. <0.05 = reject H_0 .
Statistical significance	$p < 0.05$. Result is unlikely to be due to chance.
Central Limit Theorem	Sample means follow a normal distribution regardless of population shape.
Bias (ML)	Error from model being too simple. Underfitting. High train and test error.
Variance (ML)	Error from model being too complex. Overfitting. Low train, high test error.
Bias-variance trade-off	Simplifying reduces variance but increases bias, and vice versa.
Underfitting	Model is too simple to capture patterns. High bias.
Overfitting	Model memorises training data, fails on new data. High variance.
Law of Large Numbers	More data makes probability estimates closer to the true value.

SECTION HW Homework — Submit Before Day 11

Create `day10_homework.py` and push to GitHub before midnight. Every exercise should include printed output AND a brief comment explaining what the result means in plain English. The goal today is interpretation, not just computation.

Exercise 1 — Statistics Deep Dive (Beginner)

Generate three datasets using numpy: (1) a normal distribution (mean=50, std=10, n=1000), (2) a right-skewed distribution using `np.random.exponential` (scale=20, n=1000), (3) a uniform distribution (low=0, high=100, n=1000). For each: print mean, median, std, and state in a comment which measure of central tendency is most appropriate and why.

Exercise 2 — Z-Score Outlier Detector (Beginner-Intermediate)

Create a function called `detect_outliers_zscore(data, threshold=3)` that takes a NumPy array and a z-score threshold, and returns the outlier values. Test it on this array of employee salaries: [28, 31, 29, 32, 30, 28, 500, 29, 31, 27, 480, 30, 29]. Print the outliers found and how many there are. Change the threshold to 2 and show how the result changes.

Exercise 3 — Conditional Probability on Titanic (Intermediate)

Using the cleaned Titanic dataset, compute and print the following conditional probabilities: (1) $P(\text{Survived} \mid \text{Pclass}=1)$, $P(\text{Survived} \mid \text{Pclass}=2)$, $P(\text{Survived} \mid \text{Pclass}=3)$. (2) $P(\text{Survived} \mid \text{Age} < 18)$ vs $P(\text{Survived} \mid \text{Age} \geq 18)$. (3) $P(\text{Survived} \mid \text{family_size}=1)$ vs $P(\text{Survived} \mid \text{family_size} > 1)$. Write one sentence of interpretation for each result.

Exercise 4 — A/B Test Simulation (Intermediate)

Simulate an A/B test for a website feature. Generate two groups: control group (1200 users, 15% conversion rate) and treatment group (1200 users, 18% conversion rate) using `np.random.binomial`. Run a proportions z-test (from `scipy.stats`) and report: the conversion rates, the p-value, and whether the difference is statistically significant. Then repeat with only 100 users per group and compare the p-values. Explain why more users gives a stronger result.

Hint: `from scipy.stats import proportions_ztest`. `pip install scipy` first if needed.

Exercise 5 — Bias-Variance Exploration (Intermediate)

Using NumPy polynomial fitting (`np.polyfit` and `np.polyval`), fit polynomials of degree 1, 2, 5, and 10 to a dataset of 30 points generated from $y = \sin(x) + \text{noise}$. For each model: compute the training MSE and a test MSE (use a separate test set). Print all four training and test errors in a table. Identify which degree underfits, which overfits, and which is the sweet spot.

Stretch Challenge — Tier A students

Build a complete Bayesian spam classifier from scratch using only NumPy and Pandas. Create a small dataset of 20 emails (10 spam, 10 not spam) with features like: `contains_free` (0/1), `contains_win` (0/1), `contains_click` (0/1), `sender_unknown` (0/1). Compute: $P(\text{spam})$, $P(\text{not_spam})$, $P(\text{feature}=1|\text{spam})$, $P(\text{feature}=1|\text{not_spam})$ for each feature. Then write a `predict(email_features)` function that uses Bayes theorem to classify a new email. Test it on 5 new emails and print the predicted class and probability.